TP3 — Filtre biquad

TP

DSP temps réel sur Teensy 4.x + Audio Shield v1

Guilhem PAGES **Version 1**
guilhem.pages@univ-lemans.fr **Durée : 60 min**

? Problématique

Comment implémenter une cellule de filtrage du second ordre (biquad) en temps réel, dont on balaie la fréquence de coupure en MIDI sans introduire de clic ?

Objectifs:

- Écrire une cellule biquad en forme directe II transposée (DF2T) ;
- Calculer les coefficients d'un passe-bas / passe-haut (cookbook RBJ / DAFx) ;
- Mettre à jour les coefficients sans clic (mise à jour atomique) ;
- Vérifier la réponse en fréquence à l'analyseur de spectre et discuter la quantification.

Prérequis:

- TP1 et TP2 validés (objet `AudioStream` custom) ;
- Transformée en z, fonction de transfert, filtres RII du second ordre ;
- Un PC avec Chrome (ou Edge) pour le pupitre MIDI.

Remarque *Pilotage en MIDI*

Les paramètres ne sont pas réglés par des potentiomètres mais **en MIDI depuis le PC**, par le même câble USB que l'audio. Régler *Outils* → *USB Type* → *« Serial + MIDI + Audio »*, ouvrir `outils/pupitre_midi.html` dans **Chrome**, choisir la sortie « Teensy MIDI/Audio » et le preset **TP3 – Biquad**. Chaque fader envoie un *Control Change* (0–127) que la Teensy convertit en valeur physique via `midi_map.h`.

CC	Paramètre	Plage	Loi
74	fréquence de coupure	40 – 20000 Hz	logarithmique
71	facteur de qualité Q	0,5 – 10	linéaire
81	mode	passe-bas / passe-haut	interrupteur

Comme précédemment, la partie MIDI n'est pas à étudier : vous devez simplement conserver, telles qu'elles sont fournies, les méthodes d'accès aux attributs de la classe `AudioFilterBiquadManual`.

1 Éléments théoriques

1.1 Fonction de transfert du second ordre

Un filtre numérique du second ordre est décrit par une fonction de transfert de la forme :

$$H(z) = \frac{Y(z)}{X(z)} = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}}.$$

Les coefficients b_0, b_1, b_2 définissent la partie *numérateur* du filtre, tandis que a_1 et a_2 définissent sa partie *dénominateur*.

En développant cette relation :

$$Y(z)(1 + a_1 z^{-1} + a_2 z^{-2}) = X(z)(b_0 + b_1 z^{-1} + b_2 z^{-2}),$$

puis en revenant dans le domaine temporel, on obtient l'équation aux différences :

$$y[n] = b_0 x[n] + b_1 x[n-1] + b_2 x[n-2] - a_1 y[n-1] - a_2 y[n-2].$$

Cette formulation est directement exploitable pour programmer le filtre. Cependant, elle nécessite de mémoriser les deux derniers échantillons d'entrée et les deux derniers échantillons de sortie. Une autre réalisation, la forme directe II transposée, permet d'utiliser seulement deux variables d'état.

1.2 Forme directe II transposée (DF2T)

La forme directe II transposée (*Direct Form II Transposed*, ou DF2T) réalise exactement la même fonction de transfert, mais avec une structure différente.

Elle utilise deux variables d'état, notées z_1 et z_2 :

$$\begin{aligned} y[n] &= b_0 x[n] + z_1[n], \\ z_1[n+1] &= b_1 x[n] - a_1 y[n] + z_2[n], \\ z_2[n+1] &= b_2 x[n] - a_2 y[n]. \end{aligned}$$

Les deux variables z_1 et z_2 constituent la mémoire du filtre. Elles doivent donc être conservées d'un appel à l'autre de `update()` : elles ne doivent surtout pas être réinitialisées à chaque bloc audio.

Pour chaque nouvel échantillon $x[n]$, le calcul s'effectue dans l'ordre suivant :

$$x[n] \longrightarrow y[n] \longrightarrow z_1[n+1] \longrightarrow z_2[n+1].$$

Il est important de mettre à jour les états uniquement après avoir calculé $y[n]$. Les nouvelles valeurs de z_1 et z_2 seront utilisées lors du calcul de l'échantillon suivant.

L'intérêt de cette structure est notamment de limiter le nombre de variables d'état nécessaires et de fournir une réalisation bien adaptée à l'implémentation temps réel.

En code, le traitement d'un échantillon peut donc être directement traduit par :

```
float y = b0 * x + z1;
z1 = b1 * x - a1 * y + z2;
z2 = b2 * x - a2 * y;
```

Les variables z_1 et z_2 doivent être des attributs de l'objet et non des variables locales à `update()`, car elles représentent l'état interne du filtre entre deux échantillons et entre deux blocs audio.

1.3 Calcul des coefficients : filtres “cookbook” de RBJ

Pour construire un filtre passe-bas ou passe-haut à partir de sa fréquence de coupure et de son facteur de qualité Q, on peut utiliser les équations dites “cookbook” proposées par Robert Bristow-Johnson (RBJ).

On commence par calculer la pulsation numérique :

$$\omega_0 = 2\pi \frac{f_0}{F_s},$$

où f_0 est la fréquence centrale ou de coupure souhaitée et F_s la fréquence d'échantillonnage.

On calcule ensuite :

$$\alpha = \frac{\sin(\omega_0)}{2Q}.$$

Pour un filtre passe-bas, les coefficients non normalisés sont :

$$\begin{aligned} b'_0 &= \frac{1-\cos(\omega_0)}{2}, & b'_1 &= 1-\cos(\omega_0), & b'_2 &= \frac{1-\cos(\omega_0)}{2}, \\ a_0 &= 1+\alpha, & a'_1 &= -2\cos(\omega_0), & a'_2 &= 1-\alpha. \end{aligned}$$

La fonction de transfert obtenue est alors :

$$H(z) = \frac{b'_0 + b'_1 z^{-1} + b'_2 z^{-2}}{a_0 + a'_1 z^{-1} + a'_2 z^{-2}}.$$

Or la forme utilisée précédemment suppose que le premier coefficient du dénominateur vaut 1 :

$$H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}}.$$

Il faut donc normaliser tous les coefficients par a_0 :

$$\begin{aligned} a_0 &= 1 & b_0 &= \frac{b'_0}{a_0} \\ a_1 &= \frac{a'_1}{a_0} & b_1 &= \frac{b'_1}{a_0} \\ a_2 &= \frac{a'_2}{a_0} & b_2 &= \frac{b'_2}{a_0} \end{aligned}$$

Cette normalisation est importante : les coefficients utilisés dans les équations DF2T doivent être ceux correspondant à une fonction de transfert dont le premier coefficient du dénominateur vaut exactement 1.

Pour un filtre passe-haut, les coefficients du numérateur deviennent :

$$b'_0 = \frac{1 + \cos(\omega_0)}{2}, \quad b'_1 = -(1 + \cos(\omega_0)), \quad b'_2 = \frac{1 + \cos(\omega_0)}{2}.$$

Les coefficients du dénominateur restent inchangés :

$$a_0 = 1 + \alpha, \qquad a'_1 = -2\cos(\omega_0), \qquad a'_2 = 1 - \alpha,$$

puis les cinq coefficients b_0, b_1, b_2, a_1 et a_2 sont à nouveau obtenus en les divisant par a_0.



Remarque *Quantification*

La librairie de base travaille en **virgule fixe** : l'arrondi des coefficients est sensible surtout en **basses fréquences** (pôles proches du cercle unité). Ici on calcule en **float** (FPU du Cortex-M7) : plus simple et plus stable pour l'apprentissage. On discutera l'impact du fixe.

2 Boucle de traitement DF2T

Activité 1

Dans `effect_biquad.cpp`, compléter `update()` : appliquer les trois équations DF2T échantillon par échantillon, **saturer**, et surtout **conserver l'état** z_1/z_2 d'un bloc au suivant (variables membres).

```
1 void AudioFilterBiquadManual::update(void) {
2   audio_block_t *block = receiveWritable(0);
3   if (!block) return;
4
```

```
  // Copies locales pour la boucle
  float b0 = b0_, b1 = b1_, b2 = b2_, a1 = a1_, a2 = a2_;
  float z1 = z1_, z2 = z2_;

  for (int i = 0; i < AUDIO_BLOCK_SAMPLES; i++) {
     // TODO : récupérer un échantillon du buffer et le caster en float

     // TODO : calculer le nouvel échantillon filtré puis mettre à jour l'état
     // float y = ...
     // z1 =
     // z2 =

     // TODO : saturation int16

    block->data[i] = (int16_t)y;
  }

  // Sauvegarder l'état pour le bloc suivant
  z1_ = z1; z2_ = z2;

  transmit(block);
  release(block);
}
```

3 Calcul des coefficients et mise à jour atomique

Dans un contexte temps réel, le traitement audio est exécuté périodiquement par interruption. Lors de la modification des coefficients a_i/b_i dans la fonction `AudioFilterBiquadManual::setLowpass()`, il est donc possible que le moteur audio déclenche un `update()` **pendant** la mise à jour des coefficients.

On pourrait alors avoir une situation dans laquelle certains coefficients appartiennent à l'ancien filtre tandis que les autres appartiennent au nouveau filtre. Le traitement audio utiliserait ainsi un jeu de coefficients **incohérent**, qui ne correspond à aucun des deux filtres calculés. Dans le cas d'un filtre IIR, cette situation peut provoquer une transitoire importante, voire conduire à une instabilité selon les valeurs des coefficients.

Il faut donc empêcher le traitement audio d'observer un état intermédiaire des coefficients. On cherche à rendre leur mise à jour **atomique** : le moteur audio doit observer soit l'ancien jeu complet de coefficients, soit le nouveau jeu complet, mais jamais un mélange des deux.

Pour cela, le calcul des coefficients est effectué normalement, **avant** la section critique. Seule leur copie dans les variables utilisées par le traitement audio est protégée par `AudioNoInterrupts()` :

```
  // Les coefficients sont calcules avant la section critique
  // ...

  AudioNoInterrupts();
  b0_ = b0;
  b1_ = b1;
  b2_ = b2;
  a1_ = a1;
  a2_ = a2;
  AudioInterrupts();
```

La section critique doit rester **la plus courte possible**. Il serait inutile et pénalisant de désactiver les interruptions pendant le calcul de ω_0, des fonctions trigonométriques ou des coefficients RBJ : ces

opérations sont relativement longues et pourraient perturber le traitement audio. Seule la copie des coefficients doit être protégée.

Activité 2

Compléter `setLowpass(freqHz, Q)` :

1. calculer ω_0, puis α ;
2. calculer les coefficients du filtre passe-bas selon les formules de Robert Bristow-Johnson (RBJ) ;
3. normaliser les coefficients par a_0 afin d'obtenir un dénominateur de la forme $1+a_1z^{-1}+a_2z^{-2}$;
4. copier les cinq coefficients dans les variables du filtre de façon atomique.

Pourquoi la copie des coefficients doit-elle être réalisée dans une section atomique ?

Squelette :

```
1  void AudioFilterBiquadManual::setLowpass(float freqHz, float Q) {
2    // 1) Grandeurs intermédiaires
3    float w0 = 2.0f * (float)M_PI * freqHz / FS;
4    float cw = cosf(w0);
5    float sw = sinf(w0);
6    float alpha = sw / (2.0f * Q);
7
8    // TODO 1 : calculer b0, b1, b2, a0, a1, a2
9
10   // TODO 2 : diviser b0,b1,b2,a1,a2 par a0
11
12   // TODO 3 : enregistrer les coefficients dans les attributs de la classe. Pensez à encadrer l'
       affectation des membres par AudioNoInterrupts()/AudioInterrupts()
13
14 }
```

Attention

`AudioNoInterrupts()` doit être utilisé avec parcimonie : la section critique doit être très courte. Si les interruptions sont désactivées pendant une durée trop importante, le traitement audio peut prendre du retard et des données audio peuvent être perdues.

4 Vérification spectrale

Activité 3

Envoyer un bruit ou un sweep en entrée, observer le spectre dans REW (ou l'analyseur FFT). Vérifier que la coupure suit le fader et mesurer la pente. Quelle pente attend-on pour un biquad du second ordre ?

5 Livrables

1. Objet biquad DF2T custom fonctionnel, passe-bas balayable en MIDI ;

2. Capture du spectre à 2–3 fréquences de coupure ;
3. Une phrase : pourquoi la DF2T, et pourquoi la section atomique de mise à jour ?



Remarque *Pour aller plus loin*

Ajouter un passe-haut commutable ; chaîner deux biquads (filtre du 4e ordre, -24 dB/oct) ; passer en stéréo.